

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 07 · LECTURE NOTES

Spatial Subdivision

BSP trees, k-d trees, quadtrees — the structures an engine is built on.

MODULE OUTCOMES

- Build and query a BSP tree and explain painter's-algorithm ordering.
- Build a k-d tree and analyse its queries.
- Build quadtrees and octrees and explain balancing.
- Compare uniform grids against hierarchies.
- Choose a structure from the data and the query.

1 BSP trees

EXACT VISIBILITY ORDERING FROM ANY VIEWPOINT

Each internal node stores a splitting plane, and its two children store the geometry lying on each side. Geometry that crosses the plane is *split* into two pieces, one for each side — which is where the cost comes from.

To render back-to-front from any viewpoint: at each node, determine which side the viewpoint is on, recurse into the far side, draw the node's own geometry, then recurse into the near side. **This yields an exact painter's-algorithm ordering with no per-frame sorting.**

The tree is built once, offline, and is correct for every viewpoint. That is the remarkable property, and it is why Doom and Quake were built on BSP trees — before depth buffers were cheap in hardware, this was the only way to get correct occlusion at interactive rates.

The cost is splitting. Every plane choice fragments the geometry that crosses it, and a poor sequence of choices can produce $O(n^2)$ fragments from n input polygons. Plane selection — typically a scored heuristic trading balance against split count, evaluated over a random sample of candidate planes — is essentially the whole engineering problem.

- **Solid modelling and CSG.** Boolean operations on solids are clean in a BSP formulation: each operation is a tree merge with classification at the leaves, and it sidesteps much of Module 06's boundary-walk difficulty.
- **Collision geometry.** A point, ray, or swept volume can be classified inside or outside a solid in $O(\text{depth})$, which is what the .bsp collision hulls in id-lineage engines do.
- **Portal and potentially-visible-set generation.** The leaves of a solid BSP are the convex cells between which portals are computed, in the offline visibility preprocessing step.

- **Convex decomposition.** The leaves of a solid BSP are convex by construction, which connects directly to Module 03 and Module 12's collision proxies.
- **Not for rendering order any more.** The z-buffer won that argument decisively, on cost and on simplicity. **BSP survived by changing jobs**, which is a common and underappreciated pattern for good data structures.

2 k-d trees

```
BUILD(points, depth):
    if |points| <= leaf_size: return Leaf(points)
    axis = depth mod k          # cycle x, y, z, x, ...
    m    = median along axis    # nth_element: linear time
    return Node(axis, m, BUILD(left, depth+1), BUILD(right, depth+1))
```

Build $O(n \log n)$; depth $O(\log n)$, **BALANCED BY CONSTRUCTION**
because the median splits the **COUNT**, not the space.

```
RANGE QUERY(node, box):
    if region DISJOINT from box: return          # prune
    if region CONTAINED in box:  report all      # bulk report
    else recurse into both children              # straddle
```

$O(n^{1-1/k} + m)$. In 2D: $O(\sqrt{n} + m)$. NOT logarithmic.

K-D TREES DEGRADE WITH DIMENSION, SOONER THAN EXPECTED

Orthogonal range query costs $O(n^{1-1/k} + m)$ for m reported points — $\sqrt{n}$ in two dimensions, $n^{2/3}$ in three, and approaching linear as k grows. **The square-root term surprises people who expect a tree to give logarithmic queries**; it arises because the query box boundary straddles many cells, and every straddled cell must be visited.

Nearest-neighbour search degrades similarly. Above roughly twenty dimensions a k-d tree examines most of the data and is beaten outright by a linear scan, which has perfect memory locality and no branching.

The underlying reason is that a bounding box in high dimensions touches almost everything. The volume of a high-dimensional box concentrates in its corners, the ratio of the inscribed sphere to the box vanishes, and so the pruning test almost never prunes. This is the curse of dimensionality in its most concrete form.

For graphics none of this matters — k is 2 or 3 and the bounds are good. **For feature vectors and embeddings it matters enormously**, and approximate methods take over (Module 10).

3 Quadrees, octrees, and grids

Structure	What it splits	Consequence
k-d tree	The data, at the median.	Always balanced; depth $O(\log n)$ regardless of distribution. Cell positions must be stored explicitly.
Quadtree / octree	Space, at the midpoint.	Depth depends entirely on how clustered the data is, and can be unbounded with duplicate points. In exchange, cell position is implicit — see below.
Uniform grid	Space, uniformly, with no hierarchy.	$O(1)$ lookup and $O(1)$ insert and remove, which no hierarchy matches. Fails badly on non-uniform data — either the cells are too large to prune or there are too many of them.
BVH	The data, by bounding volume.	Nodes may overlap, so a query can descend both children. The best structure for ray tracing against static geometry (CSCE 647 Module 04).
BSP	Space, by an arbitrary plane.	Fits the geometry rather than the axes, at the cost of splitting it.

Splitting the data buys balance; splitting space buys implicit addressing. That single trade organises this entire table, and knowing which side of it you need usually determines the choice.

IMPLICIT ADDRESSING IS THE QUADTREE'S REAL ADVANTAGE

A quadtree cell's position is fully determined by its path from the root, so it can be encoded as a single integer — the Morton or Z-order code, formed by interleaving the bits of the coordinates.

So cells can live in a hash table or a sorted array with no pointers at all, a parent or child code is obtained by shifting, and neighbours are found by bit arithmetic on the code. The tree structure is never materialised.

And Morton order is cache-coherent. Points sorted by Morton code are spatially local in memory, so traversals and range queries get good locality for free — **which matters more on real hardware than the asymptotic difference from a k-d tree does**, and is why GPU spatial structures are overwhelmingly Morton-based.

The cost is that depth depends on the data. A million points in one small region produces a very deep tree with mostly-empty nodes, and **exactly duplicated points produce infinite depth** unless the recursion is capped — which every implementation must do, and which is the first bug people hit.

BALANCED QUADTREES AND THE 2:1 CONSTRAINT

A quadtree is *balanced* when adjacent leaf cells differ by at most one level of subdivision — the 2:1 rule, so no cell is more than twice the size of its neighbour.

Unbalanced trees produce **hanging nodes** when converted to a mesh: a large cell's single edge meets two smaller cells' edges, so the vertex at the midpoint belongs to the small side only and the large side knows nothing about it.

That produces **visible cracks in a rendered surface and invalid elements in a simulation**. It is exactly the classic terrain LOD seam — a gap of background visible between two adjacent terrain patches at different detail levels — and it is why every terrain system has either a balance constraint or an explicit skirt or stitching pass.

Balancing costs only a constant factor in the number of cells (the refinement propagates, but geometrically decreasing), **which is a very cheap fix for a very visible problem** and should essentially always be applied.

4 Choosing

- **Ray tracing against static geometry: BVH.** Overlapping nodes are acceptable because rays are tested against bounds cheaply, and SAH-guided construction is the standard (CSCE 647 Module 04).
- **Nearest neighbour on points in two or three dimensions: k-d tree.** Balanced, simple, and the bounds are good at low k .
- **Uniformly distributed, dynamic, many insertions and removals: uniform grid.** $O(1)$ update beats any hierarchy, and particle systems and broad-phase collision are exactly this case.
- **Clustered and dynamic: loose octree or a hierarchical hash grid.** The looseness avoids objects straddling cell boundaries being pushed to the root.
- **Solid classification, CSG, and convex decomposition: BSP.**
- **Terrain, LOD, and anything with a view-dependent detail requirement: balanced quadtree.**
- **And measure on your own data.** At realistic sizes the constant factors and the memory layout usually decide this rather than the asymptotics — a flat Morton-sorted array frequently beats a pointer-based tree with better bounds.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
de Berg et al. — Computational Geometry, chapters 5, 12, 14 (range searching, BSP, quadtrees)	The analyses behind §2 and §3, including the range-query bound and the quadtree balancing argument.
Samet — Foundations of Multidimensional and Metric Data Structures	The exhaustive reference for everything in §3. The author's course slides are free and are a good substitute.
Ericson — Real-Time Collision Detection, chapters 6–8	The engine-facing treatment — grids, hierarchies, loose octrees, and the practical choices of §4.
Fabian Giesen — Morton codes and Z-order (free)	The bit arithmetic behind §3's implicit addressing, clearly explained.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Build a BSP tree from a set of polygons and render back-to-front from several viewpoints.
- 2 **Measure the fragment count** under random plane selection and under a balance-versus-splits heuristic. Report both.
- 3 Use a solid BSP to classify points inside and outside a concave shape.
- 4 Implement a k-d tree with median splitting and verify the depth is $O(\log n)$.
- 5 **Measure range query cost against n** and confirm the $\sqrt{n}$ behaviour in 2D.
- 6 Measure nearest-neighbour cost as dimension rises from 2 to 50, against a linear scan. Find the crossover.
- 7 Implement a quadtree with Morton codes and no pointers.
- 8 **Implement 2:1 balancing**, and render a terrain patch with and without it to show the cracks.
- 9 Compare a uniform grid, a quadtree, and a k-d tree on uniform and on heavily clustered data.
- 10 Profile all three for cache behaviour and relate the result to CSCE 614 and CSCE 735.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How does a BSP tree give viewpoint-independent ordering, and what does it cost?
- 2 Name four current uses of BSP trees.
- 3 Why is a k-d tree balanced by construction?
- 4 State the range query bound and say why it is not logarithmic.

- 5 Why do k-d trees fail in high dimensions?
- 6 Contrast splitting data with splitting space.
- 7 What is a Morton code and what two advantages does it give?
- 8 What is the 2:1 constraint, what does it prevent, and what does it cost?